



JL701N_PMU_介绍

JL701N_PMU_介绍 用户手册

Rev 1.2 —— 2022 年 7 月 13 日

This translated version is for reference only, and the English version shall prevail in case of any discrepancy between the translated and English versions.

版权所有 2022 杰理科技有限公司未经许可，禁止转载



目录

Chapter 1 简介	5
1.1 说明	5
1.2 名词解释	5
1.3 配置及接口路径	5
1.4 特征	6
Chapter 2 寄存器	7
2.1 说明	7
2.2 p33_io	7
Chapter 3 参数配置	13
3.1 电源参数	13
3.2 边沿唤醒参数	14
Chapter 4 唤醒	16
4.1 简介	16
4.2 接口使用	16
Chapter 5 复位	19
5.1 简介	19
5.2 接口使用	19
Chapter 6 校准	21
Chapter 7 低功耗	22
7.1 简介	22
7.2 io 配置	22
7.3 低功耗检查配置	25
7.4 接口使用	27
7.4.1 使用原则	27
7.4.2 soff	27
7.4.3 sleep	28
Chapter 8 sdk pmu 主要流程图	31
Chapter 9 客户常见问题反馈解答	33
Q1:	33



Q2: 33

ZHUHAI JIELI TECHNOLOGY CO., LTD



修改日志

版本	日期	描述
1.0	2022 / 5 / 10	JL701N 初始化版本
1.1	2022 / 5 / 30	更新边沿唤醒接口、添加低功耗时间参数说明
1.2	2022 / 7 / 13	添加低功耗使用接口、添加 sdk pmu 主要流程图



Chapter 1 简介

1.1 说明

该文档主要描述 JL701N_SDK PMU 的使用方法及开发中注意的一些问题，调试方法，具体使用方法主要包括板级配置、相关接口、PMU CPU 等，为用户进行二次开发提供参考(由于电源配置每个方案会有细微不同，请开发人员仔细查看配置方法及接口使用!!!)。

JL701N 内置 pmu，由 P11 与 P33 组成，其中 P11 为 32 位 CPU。支持包含电源、唤醒、复位、Sensor Hub 等功能，支持低功耗模式 powerdown、poweroff、softoff。

1.2 名词解释

运行状态：系统在工作，没有进入低功耗。

低功耗：包括 powerdown、poweroff、softoff

软关机：softoff，系统掉电

睡眠/sleep：powerdown/poweroff，根据应用流程选择对应模式，由系统调度进入

DCVDD：即 VDC12、VDC13，低压模块供电

IOVDD：即 VDDIO，IO 电源端，有强与弱区分，系统进入低功耗为弱驱动

MIOVDD：即 VDDIOM，运行状态下的 vddio 电源端

WIOVDD：即 VDDIOW，低功耗下的 vddio 电源端

唤醒：将 CPU 从低功耗状态下唤醒，并可以产生对应中断

1.3 配置及接口路径

电源参数：板级文件 board_xxx_cfg.c，power_param

唤醒参数：板级文件 board_xxx_cfg.c，wk_param

所有接口：include/driver/cpu/\$(CPU)/asm/power_interface.h



1.4 特征

以下电源特性参考 Datasheet，需要注意低功耗模式、运行状态下的 IOVDD 的驱动力。

Symbol	Parameter	Min	Typ	Max	Unit	Test Conditions
VBAT	Voltage Input	2.2	3.7	4.5	V	
VPWR	Charger supply Voltage	4.5	5.0	5.5	V	
Operating mode						
IOVDD	Voltage output	2.0	3.0	3.4	V	VBAT = 4.2V, 10mA loading
	Loading current	—	—	120	mA	IOVDD=3V@VBAT = 4.2V
IOVDD2	Voltage output	1.6	1.8	2.3	V	IOVDD=3V@VBAT = 4.2V
	Loading current	—	—	20	mA	Support for following IOVDD
DCVDD	Voltage output	1.0	1.25	1.4	V	IOVDD=3.0V, 10mA loading
	Loading current	—	—	100	mA	DCVDD=1.25V@IOVDD=3.0v On LDO mode
		—	—	180	mA	DCVDD=1.25V@IOVDD=3.0v On DC-DC mode
DVDD	Voltage output	0.81	1.05	1.26	V	IOVDD=3.0V
V _{LVD}	Voltage input	1.8	2.5	2.5	V	Low-Voltage Detection of IOVDD
Low Power mode						
IOVDD	Loading current	—	—	10	mA	IOVDD=3V@VBAT = 4.2V



Chapter 2 寄存器

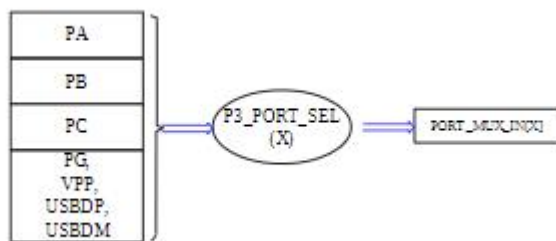
2.1 说明

本章节介绍一些 pmu 使用的寄存器，方便用户了解硬件及快速使用。

2.2 p33_io

1、p33 系统的 io 主要功能有：长按复位、边沿唤醒、脉冲计数。

2、gpio 数字输入 io 选通如下，该输入信号可以作为长按复位(数字)、边沿唤醒(数字)、脉冲计数的输入源，其中通道 0 为长按复位与边沿唤醒复用，通道 1 为脉冲计数与边沿唤醒复用。唤醒参数 wk_parm.aport[index]，其中 index 与这里的通道一一对应。



3、gpio 模拟输入，该输入信号可以作为长按复位（模拟）、边沿唤醒（模拟），通道 0 为 VBAT_DT/CHGFULL，通道 1 为 VBATCH，通道 2 为 VPWRDT，长按复位可选择通道 1 和通道 2，即 VBATCH/VPWRDT，边沿唤醒可选择通道 0~通道 2，硬件设计上已固定，唤醒参数中 wk_parm.aport[index]，其中 index 并不与这里通道对应，接口映射到对应的硬件通道。

```
{(P3_CHG_CON0[7] ? VBTDT : CHGFL), //电池欠压，充满电
VBATCH, //VPWR VBAT 比较器
VPWRDT } // VPWR 插入检测
```

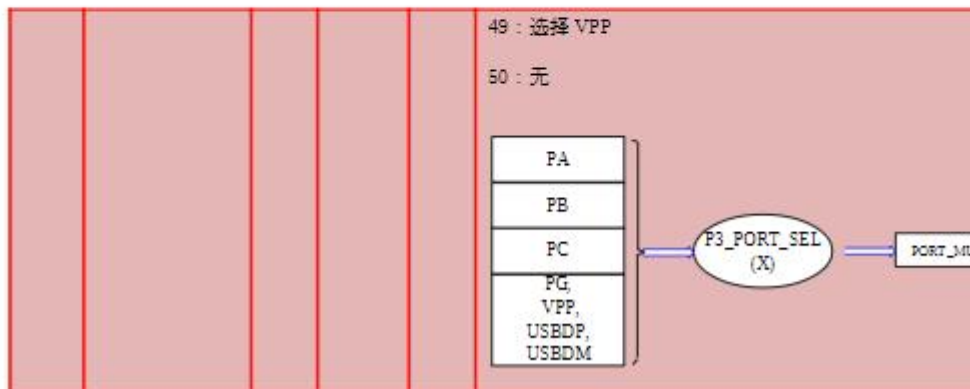
4、数字输入 io 通道选择/唤醒使能/长按复位（数字）控制/脉冲计数控制如下



27. P3_PORT_SELx (x : 0~11, 分别对应 PORT_MUX_IN0~11)

Bit	Name	RW	Default	RV	Description
7-5	RESERVED	-	-	-	预留
4-0	PORT_SEL	rw	x	-	从IO组中选取一个IO作为PORT_MUX_IN[X]. 0 : 无 1-16 : 选择 PA0-PA15 17-28 : 选择 PB0-PB11 29-37 : 选择 PC0-PC8 38-46 : 选择 PG0-PG8 47 : 选择 USBDP 48 : 选择 USBDM

10 / 13





8. P3_WKUP_EN0

Bit	Name	RW	Default	RV	Description
7-0	WKUP_EN0	rw	0	-	PORT_MUX_IN0~7 各 WKUP 的使能 0 : 不使能 1 : 使能

9. P3_WKUP_EN1

Bit	Name	RW	Default	RV	Description
7-4	RESERVED	-	-	-	预留
3-0	WKUP_EN	rw	0x0	-	PORT_MUX_IN8~11 各 WKUP 的使能 0 : 不使能 1 : 使能



1. P3_PINR_CON : 长按复位 PINR0 , 源头由 P3_PORT_SEL0 配置

3 / 13

Bit	Name	RW	Default	RV	Description
7	RESERVED	-	-	-	预留
6-4	TSEL	rw	0x2	-	长按复位时间选择： 0：长按 1s 复位 1：长按 2s 复位 2：长按 4s 复位 3：长按 8s 复位 4~7：长按 16s 复位
3	RESERVED	-	-	-	预留
2	LEVEL	rw	0	-	有效电平选择 0：低电平有效 1：高电平有效
1	RELEASE	rw	0	-	长按复位超时，复位生效时，是否立即释放系统运行： 0：需等到复位源翻转再释放系统运行 1：立即释放系统运行
0	EN	rw	1	-	长按复位使能 0：关闭长按复位功能 1：开启长按复位使能



3. P3_PCNT_CON

Bit	Name	RW	Default	RV	Description
7	PND	r	x	-	中断标志位,表示计数值溢出
6	CPND	w	0	-	清除中断标志位
5	WKUP_EN	rw	0	-	唤醒使能,中断标志位为1时,触发唤醒中断 0: 关闭唤醒功能 1: 开启唤醒功能
4	SET_WE	w	0	-	P3_PCNT_SET0/1 写使能: 0: 不使能 1: 使能 (相当 P3_PCNT_SET0/1 写入 P3_PCNT_DAT 0/1 的 kick_start, 一次有效)
3-2	SEL	rw	0x0	-	脉冲计数输入选择: 0: port_mux_in[1] 1: wclk 2: lrc_clk 3: osl_clk
1	EDGE	rw	0	-	输入脉冲反转 0: 输入脉冲保持不变 1: 输入脉冲反转
0	EN	rw	0	-	脉冲计数使能 0: 不使能 1: 使能

4、模拟输入 io 通道唤醒使能/长按复位控制（模拟）如下

18. P3_AWKUP_EN

Bit	Name	RW	Default	RV	Description
7-3	RESERVED	-	-	-	预留
2-0	AWKUP_EN	rw	0x0	-	模拟信号 awkup_in0~2 各 WKUP 的使能 0: 不使能 1: 使能



2. P3_PINR_CON1 : 长按复位 PINR1

Bit	Name	RW	Default	RV	Description
7	RESERVED	-	-	-	预留
6-4	TSEL	rw	0x2	-	长按复位时间选择： 0：长按 1s 复位 1：长按 2s 复位 2：长按 4s 复位 3：长按 8s 复位 4~7：长按 16s 复位
3	ISEL	rw	0	-	复位源头选择： 0: ldo5v_det 1: vbat_cmp_h
2	LEVEL	rw	0	-	有效电平选择 0：低电平有效

4/13

					1：高电平有效
1	REALEASE	rw	0	-	长按复位超时，复位生效时，是否立即释放系统运行： 0：需等到复位源翻转再释放系统运行 1：立即释放系统运行
0	EN	rw	1	-	长按复位使能 0：关闭长按复位功能 1：开启长按复位使能



Chapter 3 参数配置

3.1 电源参数

//以下结构体只罗列需介绍部分

```
struct low_power_param {  
    u8 config;           //低功耗使能，蓝牙&&系统空闲可进入低功耗  
    u8 osc_type;         //低功耗晶振类型，btosc/lrc  
    u32 btosc_hz;        //蓝牙晶振频率  
  
    //请配置 vddiom>vddiow  
    u8 vddiom_lev;       //vddiom  
    u8 vddiow_lev;       //vddiow  
    u8 nkeep_vddio;      //软关机状态下是否保持 vddio 的电压，默认 0 保持  
  
    u32 osc_delay_us;     //低功耗晶振起振延时，为预留配置。  
    u8 rtc_clk;           //rtc 时钟源，软关机根据此配置是否保持住相应时钟  
    u8 lpctmu_en;         //低功耗触摸，软关机根据此配置是否保持住该模块  
  
    //以下两项可配置 ram 是否进入掉电模式  
    //参考 memory mapping，ram 电源的控制变量如下  
    MEM_PWR_CPU_CON BIT(0)  
    MEM_PWR_RAM0_RAM3_CON BIT(1)  
    MEM_PWR_RAM4_RAM5_CON BIT(2)  
    MEM_PWR_RAM6_RAM7_CON BIT(3)  
    MEM_PWR_RAM8_RAM9_CON BIT(4)  
    MEM_PWR_PERIPH_CON BIT(5)
```

如配置为 MEM_PWR_RAM8_RAM_CON，则表示为电源初始化/进入低功耗将 RAM8/RAM9，进入掉电的状态。

u8 mem_init_con; //初始化 ram 电源

u8 mem_lowpower_con; //低功耗 ram 电源



//pvdd 引脚是否外接 dc/dc，外接 dc/dc 程序会将 pvdd 输出到 PB8 引脚
u8 rvdd2pvdd;

//外挂 flash gpio，使用外挂 flash 上下电接口及上电接口会操作改该引脚，默认将 flash 所有 io 配置成高阻态，**注意部分 flash 的操作使用库接口会漏电，需要重写该函数。**

void __attribute__((weak)) hw_close_flash1(void); //外挂 flash 掉电接口
void __attribute__((weak)) hw_flash1_pwr_en(); //外挂 flash 上电接口

u8 lptmr_flow; //1: 低功耗参数由用户配置 0: 默认使用库参数
//由于 sleep 对时间节点较为严格，在低功耗流程请勿添加耗时操作，例如操作系统阻塞、软件延时、打印等

//若需要在低功耗流程(sleep callback、deepsleep stash)添加相关操作，可调整低功耗时间 t2(进入低功耗)、t4(出低功耗)，单位为 us，例如添加了一段程序在进 sleep 流程，耗时 500us，则 t2+500，若在出 sleep 流程增加耗时 500us，则 t4_lrc+500，t4_btosc+500。

```
u32 t2; //进 sleep 流程 2
u32 t4_lrc; //出 sleep 流程 2 lrc
u32 t4_btosc; //出 sleep 流程 2 osc
//以下参数为参考配置
powerdown:
t1: 1200, t2: 1700, t3_lrc: 750, t4_lrc: 3165, t3_btosc: 750, t4_btosc: 1665
poweroff:
t1: 1200, t2: 1700, t3_lrc: 1250, t4_lrc: 5050, t3_btosc: 1250, t4_btosc: 4050
u8 flash_pg;
};
```

3.2 边沿唤醒参数

//最大同时支持 gpio 边沿唤醒 12 路，GPIOD、GPIOE 不支持唤醒

```
#define MAX_WAKEUP_PORT 12
```

//最大同时支持模拟信号边沿唤醒 3 路，有 LDOIN、VBTC、CHARGE_FULL

```
#define MAX_WAKEUP_ANA_PORT 3
```



```
struct port_wakeup {  
    u8 pullup_down_enable;    //内部上下拉是否使能  
    u8 edge;                  //唤醒边沿条件  
    u8 filter;                //滤波参数    gpio 边沿唤醒滤波不可配制，默认滤波为 128us  
    u8 iomap;                 //唤醒 gpio  
};
```

```
struct wakeup_param {  
    //io 唤醒  
    const struct port_wakeup *port[MAX_WAKEUP_PORT];  
    //模拟信号唤醒  
    const struct port_wakeup *aport[MAX_WAKEUP_ANA_PORT];  
};
```




Chapter 4 唤醒

4.1 简介

PMU 结构独立于系统，可以在系统睡眠或者软关机的情况下继续工作，提供多种唤醒功耗。

rtc、lvd(vddio 低压)、port edge(边沿唤醒)、pcnt overflow(计数溢出)支持唤醒低功耗状态，并产生 P33 中断，所以边沿唤醒可以当作 gpio 外部中断使用，在睡眠状态唤醒可以产生中断，在软关机状态唤醒能记录到唤醒源。

LP_TMR、LPCTMU、LP VAD 外设支持从睡眠状态下唤醒。

4.2 接口使用

头文件：include_lib/driver/cpu/\${CPU}/asm/power/lp_wakeup.h

1、边沿唤醒初始化，初始化之后默认使能

```
void power_wakeup_init(const struct wakeup_param *param)
```

2、gpio 或模拟信号边沿唤醒使能/关闭，enable 为 0 则关闭，enable 为 1 则使能。index 表示通道号，对应唤醒参数中的数组号。gpio 表示 gpio 号。

```
void power_wakeup_index_enable(u8 index, u8 enable);
```

```
void power_wakeup_gpio_enable(u8 gpio, u8 enable);
```

```
void power_awakeup_index_enable(u8 index, u8 enable);
```

```
void power_awakeup_gpio_enable(u8 gpio, u8 enable);
```

4、gpio 或模拟信号边沿类型设置，port 表示 gpio，edge 表示边沿类型

```
void power_wakeup_gpio_edge(u8 gpio, POWER_WKUP_EDGE edge);
```

```
void power_awakeup_gpio_edge(u8 gpio, POWER_WKUP_EDGE edge);
```

5、配置 gpio 或模拟信号边沿唤醒回调，该接口只能被调用一次，会被覆盖。index 为唤醒注册的通道，gpio 为 gpio 号，edge 表示边沿类型，因为模拟信号支持双边沿，所以回调函数包含了边沿信息。

```
void power_wakeup_set_callback(void (*wakeup_callback)(u8 index, u8 gpio));
```




```
void power_awakeup_set_callback(void (*wakeup_callback)(u8 index, u8 gpio, POWER_WKUP_EDGE
edge));
```

6、唤醒原因

//获取唤醒原因枚举如下

```
enum WAKEUP_REASON {
    //WAKEUP
    PWR_WK_REASON_PLUSE_CNT_OVERFLOW,    //pcnt 唤醒复位
    PWR_WK_REASON_P11,                  //P11 唤醒复位
    PWR_WK_REASON_LPCTMU,               //触摸唤醒复位
    PWR_WK_REASON_PORT_EDGE,            //数字 io 输入边沿唤醒复位
    PWR_WK_REASON_ANA_EDGE,             //模拟 io 输入边沿唤醒复位
    PWR_WK_REASON_VDDIO_LVD,            //vddio lvd 唤醒复位
    PWR_WK_REASON_EDGE_INDEX0,          //p33 index0 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX1,          //p33 index1 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX2,          //p33 index2 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX3,          //p33 index3 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX4,          //p33 index4 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX5,          //p33 index5 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX6,          //p33 index6 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX7,          //p33 index7 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX8,          //p33 index8 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX9,          //p33 index9 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX10,         //p33 index10 io 唤醒复位
    PWR_WK_REASON_EDGE_INDEX11,         //p33 index11 io 唤醒复位
    PWR_ANA_WK_REASON_FALLINIG_EDGE_LDIN, //LDO5V 上升沿唤醒复位
    PWR_ANA_WK_REASON_RISING_EDGE_LDIN,  //LDO5V 下降沿唤醒复位
    PWR_ANA_WK_REASON_FALLING_EDGE_VBAT, //VBATCH 上升沿唤醒复位
    PWR_ANA_WK_REASON_RISING_EDGE_VBAT,  //VBATCH 下降沿唤醒复位
};
```

//判断是否为 index 唤醒原因



u8 is_wakeup_source(enum WAKEUP_REASON index)



Chapter 5 复位

5.1 简介

系统复位的原因包括两种，第一种为系统复位源，第二种为自定义的复位源，例如断言软复位、异常软复位，是系统复位源的扩展，提供扩展信息、调试功能等。

5.2 接口使用

头文件：include_lib/driver/cpu/\${CPU}/asm/power/power_reset.h

//复位原因获取枚举如下

```
enum RST_REASON {  
    /*主系统*/  
    MSYS_P11_RST,           //P11 复位  
    MSYS_DVDD_POR_RST,      //DVDD 上电  
    MSYS_SOFT_RST,          //主系统软件复位  
    MSYS_P2M_RST,           //低功耗唤醒复位(softoff advance && deepsleep)  
    MSYS_POWER_RETURN,      //主系统未被复位  
  
    /*P11*/  
    P11_PVDD_POR_RST,       //pvdd 上电  
    P11_IVS_RST,            //低功耗唤醒复位(softoff legacy)  
    P11_P33_RST,            //p33 复位  
    P11_WDT_RST,            //看门狗复位  
    P11_SOFT_RST,           //软件复位  
    P11_MSYS_RST,           //主系统复位 P11  
    P11_POWER_RETURN,       //P11 系统未被复位  
  
    /*P33*/  
    P33_VDDIO_POR_RST,      //vddio 上电复位(电池/vpwr 供电)
```



```
P33_VDDIO_LVD_RST,    //vddio 低压复位、上电复位(电池/vpwr 供电)
P33_VCM_RST,          //vcm 高电平短接复位
P33_PPINR_RST,        //数字 io 输入长按复位
P33_P11_RST,          //p11 系统复位 p33, rset_mask=0
P33_SOFT_RST,         //p33 软件复位, 一般软件复位指此系统复位源, 所有系统会直接复位。
P33_PPINR1_RST,       //模拟 io 输入长按复位, 包括 charge_full、vatch、ldoint、vabt_det
P33_POWER_RETURN,     //p33 系统未被复位。
```

```
//SUB
```

```
P33_EXCEPTION_SOFT_RST,    //异常软件复位
```

```
P33_ASSERT_SOFT_RST,      //断言软件复位
```

```
};
```

```
//判断是否为 index 复位
```

```
u8 is_reset_source(enum RST_REASON index);
```



Chapter 6 校准

1、第一次上电部分电源会通过 **adc** 来校准，并写入 **vm**。需要注意校准时环境，例如 **vddio** 接了串口，会有灌电流上电造成校准不对，档校准不对时重新擦除 **vm** 即可校准。校准函数如下：

```
u8 wvdd_trim(u8 trim);
```

```
u8 pvdd_trim(u8 trim);
```

2、当发现此部分挡位与理论值相差大时可认为校准错误，一般校准错误程序跑不起来。以下挡位为 **wvdd**、**pvdd** 的理论值

WVD_S	rw	0x0	trim	Weak VDD voltage select:	
			to	0000:	0.50
			0.80	0001:	0.55
			V	0010:	0.60
				0011:	0.65
				0100:	0.70
				0101:	0.75
				0110:	0.80
				0111:	0.85

3-0	PVDD_LEVEL_L	rw	0x0	备注	pvdd step down level lowest:	
	OW			1	0000:	0.50
					0001:	0.55
					0010:	0.60
					0011:	0.65
					0100:	0.70
					0101:	0.75
					0110:	0.80
					0111:	0.85
					1000:	0.90
					1001:	0.95
					1010:	1.00
					1011:	1.05
					1100:	1.10
					1101:	1.15
					1110:	1.20
					1111:	1.25



Chapter 7 低功耗

7.1 简介

低功耗模式包括 softoff、powerdown、poweroff，以下描述从系统上来看，不包括 pmu 模块(p33 & p11)。另外在该低功耗模式的基础上可保持部分模块，例如在 powerdown 模式下可保持晶振在运行。

softoff: 所有模拟模块、数字模块都掉电。

powerdown: 所有模拟模块掉电，数字模块低电、停止运行。

poweroff: 所有模拟模块掉电，除 rvdd 外数字模块掉电、停止运行。

7.2 io 配置

不漏电原则：数字输入 die/dieh 配置状态下，io 为输入状态（cpu/外设/crossbar），外部没有确定电平有浮空漏电的危险，漏电电流不确定，请确保初始化外设/io 的状态，运行状态和低功耗模式通用。

sdk 初始化:

- 1、在 port_init 处理
- 2、所有 io die、dieh 关闭状态，上下拉关闭，过滤内置 flash/psram/pinr/sdtap

外设/crossbar/cpu 初始化:

根据不漏电原则配置 io

sleep 模式:

- 1、在板级注册进入回调函数处理 sleep_enter_callback、sleep_exit_callback
- 2、所有不使用的 io 默认配置为高阻(输入模式、上下拉关闭、数字输入关闭)，内置 flash io 由 lib 进行处理
- 3、外设/crossbar 占用的 io 如果为输入状态，保持确定电平（确保外设初始化没问题就行）
- 4、由于每个方案 io 使用情况不同，根据不漏电原则检查第 2、3 点，代码如下：



```
void sleep_enter_callback(u8 step)
{
    /* 此函数禁止添加打印 */
    if (step == 1) {
        putchar('<');
        APP_IO_DEBUG_0(A, 6);
        /* dac_power_off(); */
    } else {
        usb_iomode(1);

        gpio_set_pull_up(IO_PORT_DP, 0);
        gpio_set_pull_down(IO_PORT_DP, 0);
        gpio_set_direction(IO_PORT_DP, 1);
        gpio_set_die(IO_PORT_DP, 0);
        gpio_set_dieh(IO_PORT_DP, 0);

        gpio_set_pull_up(IO_PORT_DM, 0);
        gpio_set_pull_down(IO_PORT_DM, 0);
        gpio_set_direction(IO_PORT_DM, 1);
        gpio_set_die(IO_PORT_DM, 0);
        gpio_set_dieh(IO_PORT_DM, 0);
    }
}
```

soffoff 模式:

- 1、在板级注册进入回调函数处理 board_set_soft_poweroff
- 2、所有不使用的 io 默认配置为高阻(输入模式、上下拉关闭、数字输入关闭), 内置 flash io 由 lib 进行处理
- 3、所有外设关闭, 所有 crossbar 清除
- 4、由于每个 io 使用情况不同, 根据不漏电原则检查第 3 点, soffoff 模式根据使用外设需要保持部分 io, 代码如下



```
void board_set_soft_poweroff(void)
{
    u16 port_group[] = {
        [PORTA_GROUP] = 0xffff,
        [PORTB_GROUP] = 0xffff,
        [PORTC_GROUP] = 0xffff,
        [PORTD_GROUP] = 0xffff,
        [PORTE_GROUP] = 0xffff,
        [PORTG_GROUP] = 0xffff,
        [PORTP_GROUP] = 0x1, //
    };

    //Flash电源
    if(get_sfc_port()==0){
        port_protect(port_group, SPI0_PWR_A);
        port_protect(port_group, SPI0_CS_A);
        port_protect(port_group, SPI0_CLK_A);
        port_protect(port_group, SPI0_D0_D0_A);
        port_protect(port_group, SPI0_D1_D1_A);
        if(get_sfc_bit_mode()==4){
            port_protect(port_group, SPI0_WP_D2_A);
            port_protect(port_group, SPI0_HOLD_D3_A);
        }
    }else{
        port_protect(port_group, SPI0_PWR_B);
        port_protect(port_group, SPI0_CS_B);
        port_protect(port_group, SPI0_CLK_B);
        port_protect(port_group, SPI0_D0_D0_B);
        port_protect(port_group, SPI0_D1_D1_B);
        if(get_sfc_bit_mode()==4){
            port_protect(port_group, SPI0_WP_D2_B);
            port_protect(port_group, SPI0_HOLD_D3_B);
        }
    }
}

//power按键
#if TCFG_IOKEY_ENABLE
    port_protect(port_group, TCFG_IOKEY_POWER_ONE_PORT);
#endif

#if !(TCFG_LP_TOUCH_KEY_ENABLE && TCFG_LP_TOUCH_KEY1_EN)
    //默认唤醒io
    port_protect(port_group, IO_PORTB_01);
#endif

#if (TCFG_TEST_BOX_ENABLE || TCFG_CHARGESTORE_ENABLE || TCFG_ANC_BOX_ENABLE || TCFG_UMIDIGI_BOX_ENABLE)
    power_wakeup_index_disable(2);
#endif

mask_io_cfg();

gpio_dir(GPIOA, 0, 16, port_group[PORTA_GROUP], GPIO_OR);
gpio_set_pu(GPIOA, 0, 16, ~port_group[PORTA_GROUP], GPIO_AND);
gpio_set_pd(GPIOA, 0, 16, ~port_group[PORTA_GROUP], GPIO_AND);
gpio_die(GPIOA, 0, 16, ~port_group[PORTA_GROUP], GPIO_AND);
gpio_dieh(GPIOA, 0, 16, ~port_group[PORTA_GROUP], GPIO_AND);

gpio_dir(GPIOB, 0, 16, port_group[PORTB_GROUP], GPIO_OR);
gpio_set_pu(GPIOB, 0, 16, ~port_group[PORTB_GROUP], GPIO_AND);
gpio_set_pd(GPIOB, 0, 16, ~port_group[PORTB_GROUP], GPIO_AND);
gpio_die(GPIOB, 0, 16, ~port_group[PORTB_GROUP], GPIO_AND);
gpio_dieh(GPIOB, 0, 16, ~port_group[PORTB_GROUP], GPIO_AND);

gpio_dir(GPIOC, 0, 16, port_group[PORTC_GROUP], GPIO_OR);
gpio_set_pu(GPIOC, 0, 16, ~port_group[PORTC_GROUP], GPIO_AND);
gpio_set_pd(GPIOC, 0, 16, ~port_group[PORTC_GROUP], GPIO_AND);
gpio_die(GPIOC, 0, 16, ~port_group[PORTC_GROUP], GPIO_AND);
gpio_dieh(GPIOC, 0, 16, ~port_group[PORTC_GROUP], GPIO_AND);

gpio_dir(GPIOD, 0, 16, port_group[PORTD_GROUP], GPIO_OR);
gpio_set_pu(GPIOD, 0, 16, ~port_group[PORTD_GROUP], GPIO_AND);
gpio_set_pd(GPIOD, 0, 16, ~port_group[PORTD_GROUP], GPIO_AND);
gpio_die(GPIOD, 0, 16, ~port_group[PORTD_GROUP], GPIO_AND);
gpio_dieh(GPIOD, 0, 16, ~port_group[PORTD_GROUP], GPIO_AND);

gpio_dir(GPIOE, 0, 16, port_group[PORTE_GROUP], GPIO_OR);
gpio_set_pu(GPIOE, 0, 16, ~port_group[PORTE_GROUP], GPIO_AND);
gpio_set_pd(GPIOE, 0, 16, ~port_group[PORTE_GROUP], GPIO_AND);
gpio_die(GPIOE, 0, 16, ~port_group[PORTE_GROUP], GPIO_AND);
gpio_dieh(GPIOE, 0, 16, ~port_group[PORTE_GROUP], GPIO_AND);

gpio_dir(GPIOG, 0, 16, port_group[PORTG_GROUP], GPIO_OR);
gpio_set_pu(GPIOG, 0, 16, ~port_group[PORTG_GROUP], GPIO_AND);
gpio_set_pd(GPIOG, 0, 16, ~port_group[PORTG_GROUP], GPIO_AND);
gpio_die(GPIOG, 0, 16, ~port_group[PORTG_GROUP], GPIO_AND);
gpio_dieh(GPIOG, 0, 16, ~port_group[PORTG_GROUP], GPIO_AND);

gpio_dir(GPIOP, 0, 1, port_group[PORTP_GROUP], GPIO_OR);
gpio_set_pu(GPIOP, 0, 1, ~port_group[PORTP_GROUP], GPIO_AND);
gpio_set_pd(GPIOP, 0, 1, ~port_group[PORTP_GROUP], GPIO_AND);
gpio_die(GPIOP, 0, 1, ~port_group[PORTP_GROUP], GPIO_AND);
gpio_dieh(GPIOP, 0, 1, ~port_group[PORTP_GROUP], GPIO_AND);

gpio_set_pull_up(IO_PORT_DP, 0);
gpio_set_pull_down(IO_PORT_DP, 0);
gpio_set_direction(IO_PORT_DP, 1);
gpio_set_die(IO_PORT_DP, 0);
gpio_set_dieh(IO_PORT_DP, 0);

gpio_set_pull_up(IO_PORT_DM, 0);
gpio_set_pull_down(IO_PORT_DM, 0);
gpio_set_direction(IO_PORT_DM, 1);
gpio_set_die(IO_PORT_DM, 0);
gpio_set_dieh(IO_PORT_DM, 0);

dac_power_off();
}
```




7.3 低功耗检查配置

在文件 `cpu/${CPU}/power_api.c`。

- 1、提供以下几个打印接口用于 debug

```
void pmut_put_u8hex(u8 dat);  
void pmu_put_u16hex(u16 dat);  
void pmu_put_u32hex(u32 dat);  
void pmu_put_u64hex(u64 dat);  
void pmu_put_buf(const u8 *buf, int len);
```

- 2、初始化 `uart0` 为 `rc16m` 时钟，由于 `rc16m` 受温度及电压影响，请在 `app_main` 函数调用 `void config_p11_baud()` 进行 `rc` 频率检测，源码如下：



```
AT_VOLATILE_RAM_CODE
u32 capture_rch_freq()
{
    u32 sys_div_bak;
    u32 frch;

    //clock set to rc, sys div = 0
    JL_CLOCK->CLK_CON0 |= BIT(0);
    SFR(JL_CLOCK->CLK_CON1, 8, 3, 3);
    sys_div_bak = JL_CLOCK->SYS_DIV;
    JL_CLOCK->SYS_DIV = 0;

    //主时钟
    GPCNT_CLOCK_IN(GPCNT_CLOCK_IN_HSB);
    GPCNT_GSS(GPCNT_GSS_CLOCK_IN);
    GPCNT_CSS(GPCNT_CSS_PLL);
    GPCNT_GTS(6);
    GPCNT_CLR_PEND(1);
    GPCNT_EN(1);
    while (!(JL_GPCNT->CON & BIT(7)));
    GPCNT_EN(0);

    frch = (32 * (1 << 6) * (clk_get("pll") / 1000)) / JL_GPCNT->NUM;

    //recover sys_div, close set to pll
    JL_CLOCK->SYS_DIV = sys_div_bak;
    SFR(JL_CLOCK->CLK_CON1, 8, 3, 6);

    return frch;
}

AT_VOLATILE_RAM_CODE
void config_pll_baud()
{
    rch_freq = capture_rch_freq();

    printf("frch: %d\n", rch_freq);

    M2P_RCH_FREQ_L = rch_freq & 0xff;
    M2P_RCH_FREQ_H = ((rch_freq & 0xff00) >> 8);
}
```

3、由于需要使用 rc16m 打印，板级配置波特率如下：

```
#define TCFG_UART0_BAUDRATE    115200                //串口波特率配置
```

4、在对应 debug 点检查 io/外设状态，初始化串口 0，时钟源为 rc16m，IO 为 PA05，波特率为 115200，并且 debug 点代码只能跑 ram。

//powerdown/poweroff 检查点

u8 sleep_safety_check(void)

//softoff 检查点

u8 soff_safety_check(void)

//该函数会降所有 io 的寄存器打印出来，包括方向、上下拉、数字输入控制位，检查对应的 io 状态是否正确，源码如下

void check_gpio_safety()



```
#pragma bss_seg(".volatile_ram_code")
#pragma data_seg(".volatile_ram_code")
#pragma code_seg(".volatile_ram_code")
#pragma const_seg(".volatile_ram_code")
#pragma str_literal_override(".volatile_ram_code")

int pmu_puts(const char *s);
void pmu_put_u16hex(u16 dat);

u16 r3_gpio_check = 0x3;

char *port_string[] = {
    "GPIOA",
    "GPIOB",
    "GPIOC",
    "GPIOD",
    "GPIOE",
    "GPIOG",
    "GPIOH",
    "GPIOI",
};

u32 gpio_regs[] = {
    (u32) JL_PORTA,
    (u32) JL_PORTB,
    (u32) JL_PORTC,
    (u32) JL_PORTD,
    (u32) JL_PORTE,
    (u32) JL_PORTF,
    (u32) JL_PORTG,
    (u32) JL_PORTP,
};

__NOINLINE
__AT_VOLATILE_RAM_CODE
void check_gpio_safety()
{
    struct gpio_reg *gpio;
    /*pmu_puts("check_gpio_safety");*/

    //general
    for (u8 i = 0; i < 7; i++) {
        gpio = gpio_regs[i];
        pmu_puts("\n");
        pmu_puts(port_string[i]);

        pmu_puts(" dir-");
        pmu_put_u16hex(gpio->dir);

        pmu_puts(" pu-");
        pmu_put_u16hex(gpio->pu);

        pmu_puts(" pd-");
        pmu_put_u16hex(gpio->pd);

        pmu_puts(" die-");
        pmu_put_u16hex(gpio->die);

        pmu_puts(" dieh-");
        pmu_put_u16hex(gpio->dieh);
    }

    u16 control_by_p11 = P11_PORT->PB_SEL;
    pmu_puts("\n");
    pmu_puts("P11PORT");

    pmu_puts(" dir-");
    pmu_put_u16hex(P11_PORT->PB_DIR & control_by_p11);

    pmu_puts(" pu-");
    pmu_put_u16hex(P11_PORT->PB_PU & control_by_p11);

    pmu_puts(" pd-");
    pmu_put_u16hex(P11_PORT->PB_PD & control_by_p11);

    pmu_puts(" die-");
    pmu_put_u16hex(P11_PORT->PB_DIE & control_by_p11);

    pmu_puts(" dieh-");
    pmu_put_u16hex(P11_PORT->PB_DIEH & control_by_p11);

    pmu_puts(" con-");
    pmu_put_u16hex(control_by_p11);

    pmu_puts("\n");
}
```

7.4 接口使用

7.4.1 使用原则

所有在 sleep、soff 模式提供的回调接口均在系统临界，请勿使用操作系统提供的接口。此外 sleep 对时间节点较为严格，在低功耗流程请勿添加耗时操作，例如操作系统阻塞、软件延时、打印等。

7.4.2 soff

1、注册软关机回调函数，在 soff 模式回调该接口，用户可以在该函数处理设备的关机状态，包括 gpio、soff_flag、dac 等，通用外设例如 iic、spi 等设备由驱动库进行处理，app 程序不需要处理。



```
void power_set_callback(u8 mode, void (*powerdown_enter)(u8 step), void (*powerdown_exit)(u32),  
void (*soft_poweroff_enter)(void));
```

其中 soft_poweroff_enter 为软关机回调函数，sdk 中的示例代码在“io 配置”小节描述。

2、soff_flag

soff_flag 是用于配置软关机起来的状态信息，包括 io、启动信息等，由于软关机状态会锁住 io，开机之后在 rom 会释放一部分 io，在 sdk 会将所有 io 释放，若想保持 io 的状态，必须在释放 io 之前配置状态，rom 会根据 soff_flag 配置 io 然后再释放 io，如下所示：

```
static void mask_io_cfg()  
{  
    struct boot_soft_flag_t boot_soft_flag = {0};  
    boot_soft_flag.flag0.boot_ctrl.wdt_dis = 0;  
    boot_soft_flag.flag0.boot_ctrl.poweroff = 0;  
    boot_soft_flag.flag0.boot_ctrl.is_port_b = JL_IOMAP->CON0 & BIT(16) ? 1 : 0;  
  
    boot_soft_flag.flag1.misc.usbdm = SOFTFLAG_HIGH_RESISTANCE;  
    boot_soft_flag.flag1.misc.usbdp = SOFTFLAG_HIGH_RESISTANCE;  
  
    boot_soft_flag.flag1.misc.uart_key_port = 0;  
    boot_soft_flag.flag1.misc.ldoin = SOFTFLAG_HIGH_RESISTANCE;  
  
    boot_soft_flag.flag2.pg2_pg3.pg2 = SOFTFLAG_HIGH_RESISTANCE;  
    boot_soft_flag.flag2.pg2_pg3.pg3 = SOFTFLAG_HIGH_RESISTANCE;  
  
    boot_soft_flag.flag3.pg4_res.pg4 = SOFTFLAG_HIGH_RESISTANCE;  
  
    boot_soft_flag.flag4.fast_boot_ctrl.fast_boot = 0;  
    boot_soft_flag.flag4.fast_boot_ctrl.flash_stable_delay_sel = 0;  
  
    mask_softflag_config(&boot_soft_flag);  
}
```

7.4.3 sleep

当操作系统与蓝牙空闲时可进入 sleep 模式，即所有任务空闲且蓝牙 idle，sleep 接口调用流程：

1、low_power_sys_is_idle：查询所有注册模块是否可进入 sleep 模式，使用 REGISTER_LP_TARGET 注册，该结构如下所示：

```
enum LOW_POWER_LEVEL {  
    LOW_POWER_MODE_LIGHT_SLEEP = 1,  
    LOW_POWER_MODE_SLEEP,
```



```
LOW_POWER_MODE_DEEP_SLEEP,  
};
```

```
typedef u8(*idle_handler_t)(void);  
typedef enum LOW_POWER_LEVEL(*level_handler_t)(void);
```

```
typedef u8(*idle_handler_t)(void);
```

```
struct lp_target {  
    char *name;                //模块名称  
    level_handler_t level;      //进入 sleep 等级，light_sleep、sleep、deepsleep  
    idle_handler_t is_idle;     //是否可进入 sleep  
};
```

```
#define REGISTER_LP_TARGET(target) \  
    const struct lp_target target sec(.lp_target)
```

2、low_power_sys_request_enter: 满足操作系统与蓝牙可进入低功耗之后，主动请求关闭剩余的模块，使用 REGISTER_LP_REQUEST 注册，该接口的使用主要是用于关闭流程比较复杂且不需要频繁开关的模块，例如外置 flash、dac 模块。

```
struct lp_request {  
    char *name;                //模块名称  
    u8(*request_enter)(u32 timeout); //请求进入，返回值 1：模块还未关闭，0：模块已关闭  
    u8(*request_exit)(u32 timeout);  //请求退出  
};
```

```
#define REGISTER_LP_REQUEST(target) \  
    const struct lp_request target sec(.lp_request)
```

```
extern const struct lp_request lp_request_begin[];
```

```
extern const struct lp_request lp_request_end[];
```



```
#define list_for_each_lp_request(p) \  
    for (p = lp_request_begin; p < lp_request_end; p++)
```

3、开始进入低功耗流程回调用户接口，用户注册如下：

```
void power_set_callback(u8 mode, void (*powerdown_enter)(u8 step), void (*powerdown_exit)(u32),  
void (*soft_poweroff_enter)(void));
```

其中 powerdown_enter 为 sleep 进入回调函数。

4、sleep

5、tmr/io wkup

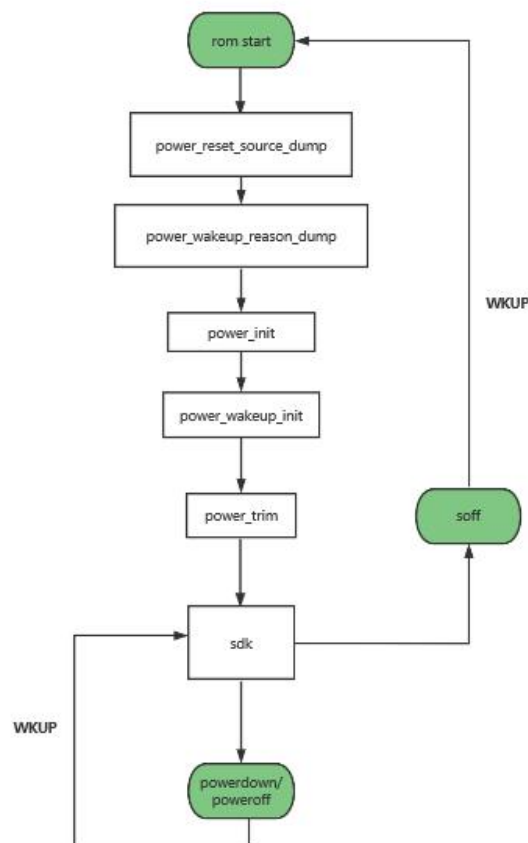
6、退出低功耗流程回调用户接口

```
void power_set_callback(u8 mode, void (*powerdown_enter)(u8 step), void (*powerdown_exit)(u32),  
void (*soft_poweroff_enter)(void));
```

其中 powerdown_exit 为 sleep 回调函数。



Chapter 8 sdk pmu 主要流程图



- 1、上电首先会对复位源、唤醒源检查及记录
- 2、电源初始化与唤醒初始化
- 3、低功耗流程，包括 powerdown/poweroff/soff



ZHUHAI JIELI TECHNOLOGY CO., LTD



Chapter 9 客户常见问题反馈解答

Q1:

Q2:
